

UNIVERSIDAD NACIONAL DE SAN CRISTOBAL DE HUAMANGA
FACULTAD DE INGENIERIA DE MINAS, GEOLOGIA Y CIVIL
ESCUELA PROFESIONAL DE INGENIERIA DE SISTEMAS

INFORME DE INVESTIGACION Y DOCUMENTACION TECNICA

**"Aura Marketplace: plataforma web de comercio electronico inteligente
con arquitectura hexagonal, frontend modular y agente conversacional"**

Curso: Ingenieria de Software / Arquitectura de Sistemas

Proyecto: Aura Marketplace

Presentado por: _____

Docente: _____

Ayacucho - Peru

2026

Dedicatoria

Dedico este trabajo a mi familia, por el apoyo constante durante el proceso de análisis, diseño, implementación y documentación del sistema. Su acompañamiento permitió sostener el esfuerzo necesario para convertir una idea de marketplace inteligente en una propuesta técnica estructurada.

Asimismo, dedico el presente informe a los estudiantes y docentes que comprenden que la ingeniería de software no termina en la programación, sino que exige justificar las decisiones, documentar la arquitectura, evaluar la calidad y dejar evidencia suficiente para que el sistema pueda evolucionar.

Agradecimiento

Agradezco a los docentes de la Escuela Profesional de Ingeniería de Sistemas por impulsar una visión rigurosa del desarrollo de software. La elaboración de este informe se apoya en conocimientos de arquitectura, bases de datos, seguridad, metodologías ágiles, frontend, backend, pruebas y documentación técnica.

También se expresa agradecimiento a quienes participaron en la revisión del proyecto Aura Marketplace, ya que sus observaciones permitieron ordenar los requerimientos, consolidar los diagramas anexos y convertir los hallazgos del repositorio en un documento académico de más de cien páginas.

Resumen

El presente informe desarrolla la documentación integral del sistema Aura Marketplace, una plataforma web de comercio electrónico inteligente que combina una interfaz SPA en React con un backend NestJS basado en arquitectura hexagonal, persistencia relacional en PostgreSQL mediante Prisma y un agente conversacional capaz de interpretar instrucciones en lenguaje natural.

El objetivo principal del documento es explicar, con enfoque académico y técnico, el marco teórico, la arquitectura general, el diseño del backend, el diseño del frontend, la metodología Scrum aplicada y los anexos de ingeniería de software. Para ello se revisó el documento de ejemplo proporcionado, el repositorio del sistema, las especificaciones, los archivos de diseño, el esquema de base de datos y los diagramas entregados como imágenes.

Como resultado, se presenta una propuesta documentada que supera las cien páginas e integra capítulos de planteamiento del problema, marco teórico, material y métodos, resultados de diseño e implementación, conclusiones, recomendaciones y anexos. El informe enfatiza la aplicación de Specs Driven Development (SDD), la separación por capas L01-L05, la trazabilidad de requisitos y la coherencia entre frontend, backend, base de datos e integraciones externas.

- Palabras clave: Aura Marketplace, arquitectura hexagonal, React, NestJS, Prisma, Scrum, SDD, agente inteligente.

Abstract

This report presents the complete technical and academic documentation for Aura Marketplace, an intelligent e-commerce web platform that combines a React single page application, a NestJS backend based on hexagonal architecture, PostgreSQL persistence through Prisma, and a conversational agent capable of interpreting natural language instructions.

The document explains the theoretical framework, the general system architecture, backend and frontend architecture, the Scrum methodology, and the software engineering appendices. The analysis is based on the provided reference PDF, the actual project repository, specification documents, design documents, database schema, deployment files, and architectural diagrams supplied as annex images.

The resulting Word document is structured as a thesis-like report with more than one hundred pages, including front matter, problem statement, theoretical framework, methodology, technical results, conclusions, recommendations, bibliography, and annexes. Special attention is given to Specs Driven Development, the L01-L05 backend layering model, requirements traceability, and the alignment between frontend, backend, database, and external integrations.

- Keywords: Aura Marketplace, hexagonal architecture, React, NestJS, Prisma, Scrum, SDD, intelligent agent.

Contenido

La organización del documento sigue la estructura observada en el archivo de ejemplo: páginas preliminares, introducción, cinco capítulos principales, referencias bibliográficas y anexos. La diferencia principal es que el contenido se adapta al sistema Aura Marketplace y a los diagramas técnicos proporcionados.

Sección	Descripción
Introducción	Contexto del proyecto, alcance y estructura del informe
Capítulo I	Planteamiento del problema, objetivos, hipótesis y justificación
Capítulo II	Marco teórico: SDD, arquitectura, backend, frontend y Scrum
Capítulo III	Material y métodos: enfoque aplicado, variables y gestión Scrum
Capítulo IV	Resultados: arquitectura, diseño, implementación, pruebas y despliegue
Capítulo V	Conclusiones y recomendaciones
Referencias	Bibliografía usada para sustentar la documentación técnica
Anexos	SDD, backlog, matriz, diagramas, arquitectura y evidencias del proyecto

Lista de figuras y tablas

Las figuras incluidas en los anexos corresponden a los diagramas proporcionados por el usuario y a los artefactos generados en el repositorio. Las tablas sintetizan requisitos, reglas de negocio, trazabilidad, backlog, sprint planning, arquitectura por capas, endpoints, entidades de datos y escenarios SDD.

Elemento	Contenido
Figura A1	Arquitectura general del backend y dependencias externas
Figura A2	Arquitectura modular del frontend React
Figura A3	Boundaries de backend, infraestructura y datos
Figura A4	Modelo entidad-relacion del marketplace
Figura A5	Stack tecnologico y despliegue conectado
Tablas	Matriz de consistencia, backlog, requisitos, reglas, pruebas y SDD

Introduccion

La digitalización del comercio ha transformado la manera en que compradores, vendedores y administradores interactúan dentro de un ecosistema transaccional. En este escenario, Aura Marketplace se plantea como una plataforma que no solo publica productos y procesa compras, sino que también reduce la fricción mediante un agente inteligente capaz de interpretar instrucciones en lenguaje natural.

El sistema se organiza como un monorepo con frontend en React, backend en NestJS, base de datos PostgreSQL y servicios externos para autenticación, pagos, correo, almacenamiento, cache e inteligencia artificial. Esta composición exige una documentación extensa que explique tanto el fundamento teórico como la arquitectura concreta de implementación.

El documento toma como referencia formal el archivo TrabajoEjemplo.pdf, cuyo patrón académico contiene portada, resumen, abstract, capítulos metodológicos, resultados, conclusiones y anexos. Sobre esa base, el presente informe desarrolla una versión orientada al proyecto Aura Marketplace, incorporando los diagramas anexos enviados por el usuario.

Capítulo I

1.1 Diagnostico y enunciado del problema

Los marketplaces tradicionales obligan al usuario a transformar su necesidad en filtros, categorías, palabras clave y acciones repetitivas. Esta conversión entre intención humana e interacción gráfica produce fricción, especialmente para usuarios con baja experiencia digital, limitaciones visuales o preferencia por interfaces conversacionales.

En Aura Marketplace, el problema se observa como una brecha entre el lenguaje natural del comprador y los flujos tradicionales de exploración, comparación, carrito y pago. Si el sistema no interpreta correctamente esa intención, el usuario debe navegar manualmente, repetir búsquedas o abandonar la compra.

Adicionalmente, la implementación de un sistema de comercio electrónico inteligente exige resolver problemas de seguridad, trazabilidad, integridad transaccional, control de roles, persistencia confiable y escalabilidad. Por ello, la solución no puede limitarse a una interfaz atractiva; requiere arquitectura backend, frontend, datos, integraciones y metodología de desarrollo claramente documentadas.

- Fricción entre la intención del usuario y las acciones de compra.
- Necesidad de organizar el backend para aislar dominio, aplicación e infraestructura.
- Necesidad de un frontend modular que soporte roles, carrito, catálogo, checkout y agente.
- Necesidad de anexos técnicos que evidencien SDD, ERD, despliegue y decisiones arquitectónicas.

1.2 Problema general

En el contexto de Aura Marketplace, determinar como documentar, diseñar e implementar una plataforma de comercio electrónico inteligente que permita a compradores, vendedores y administradores operar con seguridad, baja fricción y trazabilidad técnica. Se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de planteamiento del problema, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Se mantiene correspondencia con los objetivos funcionales y no funcionales.
- Se documenta la relación entre el problema y las decisiones de arquitectura.
- Se reserva evidencia técnica detallada para los anexos.

1.3 Problemas específicos

En el contexto de Aura Marketplace, identificar las dificultades asociadas al modelado de requisitos, la comunicación frontend-backend, la persistencia transaccional, la seguridad de sesiones, la integración de servicios externos y la aplicación de SDD en los anexos. se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de planteamiento del problema, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Se mantiene correspondencia con los objetivos funcionales y no funcionales.
- Se documenta la relación entre el problema y las decisiones de arquitectura.
- Se reserva evidencia técnica detallada para los anexos.

1.4 Objetivo general

En el contexto de Aura Marketplace, desarrollar una documentación integral del sistema Aura Marketplace que explique el marco teórico, la metodología Scrum, la arquitectura general, el backend, el frontend, la base de datos, las integraciones, las pruebas y los anexos técnicos. se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de planteamiento del problema, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Se mantiene correspondencia con los objetivos funcionales y no funcionales.
- Se documenta la relación entre el problema y las decisiones de arquitectura.
- Se reserva evidencia técnica detallada para los anexos.

1.5 Objetivos específicos

En el contexto de Aura Marketplace, organizar el informe en capítulos académicos, describir la aplicación de SDD, justificar la arquitectura hexagonal, explicar la modularidad del frontend, documentar la base de datos y elaborar anexos con diagramas y tablas de soporte. se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de planteamiento del problema, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Se mantiene correspondencia con los objetivos funcionales y no funcionales.
- Se documenta la relación entre el problema y las decisiones de arquitectura.
- Se reserva evidencia técnica detallada para los anexos.

1.6 Hipotesis

En el contexto de Aura Marketplace, si el sistema se documenta mediante una estructura academica basada en Scrum, SDD y arquitectura hexagonal, entonces se obtiene un informe verificable que evidencia la coherencia entre requisitos, diseño e implementación. se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de planteamiento del problema, el proyecto requiere separar los conceptos teoricos de las decisiones de implementación. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingeniería: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Se mantiene correspondencia con los objetivos funcionales y no funcionales.
- Se documenta la relacion entre el problema y las decisiones de arquitectura.
- Se reserva evidencia tecnica detallada para los anexos.

1.7 Justificación técnica

En el contexto de Aura Marketplace, sustentar que el proyecto requiere puertos, adaptadores, DTOs, guards, repositorios, stores de estado, validaciones, integraciones y despliegue separados para mantener la evolución del producto. se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de planteamiento del problema, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Se mantiene correspondencia con los objetivos funcionales y no funcionales.
- Se documenta la relación entre el problema y las decisiones de arquitectura.
- Se reserva evidencia técnica detallada para los anexos.

1.8 Justificación social y académica

En el contexto de Aura Marketplace, demostrar que una plataforma con agente conversacional puede ampliar el acceso al comercio electrónico, reducir barreras de uso y servir como caso de estudio para ingeniería de software. Se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de planteamiento del problema, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Se mantiene correspondencia con los objetivos funcionales y no funcionales.
- Se documenta la relación entre el problema y las decisiones de arquitectura.
- Se reserva evidencia técnica detallada para los anexos.

1.9 Delimitación del sistema

En el contexto de Aura Marketplace, acotar el informe al monorepo existente, las imágenes entregadas y los documentos de especificación y diseño ubicados en las carpetas specs y design. se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de planteamiento del problema, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Se mantiene correspondencia con los objetivos funcionales y no funcionales.
- Se documenta la relación entre el problema y las decisiones de arquitectura.
- Se reserva evidencia técnica detallada para los anexos.

1.10 Limitaciones

En el contexto de Aura Marketplace, reconocer que las métricas de carga, accesibilidad total y disponibilidad dependen de pruebas futuras y de la configuración final de servicios como Render, Vercel, Neon, Mercado Pago y Cloudinary. se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de planteamiento del problema, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Se mantiene correspondencia con los objetivos funcionales y no funcionales.
- Se documenta la relación entre el problema y las decisiones de arquitectura.
- Se reserva evidencia técnica detallada para los anexos.

Capítulo II

Este capítulo se desarrolla con base bibliográfica y citas en formato APA 7. Las citas textuales se emplean de manera breve; las demás ideas se presentan como parafrasis académicas con su respectiva fuente.

2.1 Antecedentes de la investigación

La ingeniería de software moderna sostiene que un sistema debe comprenderse como un producto sociotécnico: satisface necesidades humanas, pero se materializa mediante procesos, modelos, código, pruebas y despliegue. Sommerville (2016) explica que los sistemas intensivos en software no pueden evaluarse solo por su ejecución, sino también por su capacidad de evolucionar, mantenerse y operar con restricciones de calidad. En Aura Marketplace, este antecedente justifica que el informe no se limite a pantallas o endpoints, sino que documente visión, requisitos, arquitectura, base de datos, seguridad y metodología.

Desde una perspectiva complementaria, Pressman y Maxim (2020) señalan que la calidad se construye durante todo el proceso y no solamente al final. Por ello, el marco teórico vincula SDD, Scrum, arquitectura hexagonal, frontend modular y pruebas como prácticas integradas. La plataforma estudiada requiere esa integración porque une comercio electrónico, agentes conversacionales, pagos externos, autenticación, inventario, roles y auditoría.

- Base bibliográfica: Sommerville (2016) y Pressman y Maxim (2020).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.2 Specs Driven Development (SDD)

Specs Driven Development se entiende en este informe como un enfoque en el que las especificaciones son la fuente de verdad del desarrollo. La norma IEEE/ISO/IEC 29148 establece procesos y productos para la ingeniería de requisitos a lo largo del ciclo de vida del software. En cita textual breve, dicha norma alude a "software products and services throughout the life cycle" (IEEE/ISO/IEC, 2018, secc. Scope).

Aplicado a Aura Marketplace, SDD significa que los archivos de la carpeta specs no son documentos decorativos: Vision, Stakeholders, Glosario, Alcance, Objetivos, Reglas de Negocio, Requisitos Funcionales, Requisitos No Funcionales, Actores, Casos de Uso, Criterios de Aceptación y Matriz de Trazabilidad orientan el diseño de módulos, endpoints, entidades, stores, pruebas y anexos. Esta relación convierte cada cambio técnico en una decisión justificable.

- Base bibliográfica: IEEE/ISO/IEC (2018).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.3 Requisitos de software y trazabilidad

La trazabilidad permite seguir el recorrido de una necesidad desde su formulación hasta su verificación. IEEE/ISO/IEC 29148 (2018) destaca la importancia de atributos, características y aplicación iterativa de los procesos de requisitos. En proyectos con múltiples capas, la trazabilidad evita que los módulos se implementen por intuición y obliga a demostrar que una función responde a una necesidad identificada.

En Aura Marketplace, esta base teórica se evidencia cuando un objetivo como reducir la fricción de compra se transforma en requisitos del agente, reglas de confirmación, criterios de aceptación, controladores REST, stores de estado, componentes de UI y pruebas E2E. El anexo SDD debe mostrar esta cadena, porque el desarrollo guiado por especificaciones solo es válido si cada artefacto se puede rastrear.

- Base bibliográfica: IEEE/ISO/IEC (2018) y Sommerville (2016).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.4 Plataforma web de comercio electrónico

Un marketplace digital combina catálogo, identidad, transacciones, confianza, pagos, soporte y administración. Pressman y Maxim (2020) recomiendan modelar estas funciones como requisitos verificables y no como listas informales de deseos. En Aura Marketplace, la capa de comercio electrónico se expresa mediante publicaciones, categorías, inventario, carrito, órdenes, pagos, favoritos, reseñas, promociones y notificaciones.

La importancia teórica del comercio electrónico en este informe radica en que cada flujo implica restricciones de datos y seguridad. Una compra no es solamente una página de checkout: requiere autenticación, validación de stock, persistencia atómica, integración con pasarela de pago, registro histórico y notificación. Por ello, el sistema necesita arquitectura por capas y trazabilidad documental.

- Base bibliográfica: Pressman y Maxim (2020) y Sommerville (2016).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.5 Agente inteligente conversacional

El agente conversacional se fundamenta en la necesidad de reducir la distancia entre la intención humana y la acción funcional del sistema. Desde el punto de vista de requisitos, esta capacidad debe definirse mediante entradas, salidas, reglas de negocio, restricciones de seguridad y criterios de aceptación (IEEE/ISO/IEC, 2018). La interacción en lenguaje natural no reemplaza la arquitectura; la exige con mayor claridad.

En Aura Marketplace, el agente no debe operar como un módulo aislado sin control. Su interpretación de intenciones se conecta con catálogo, carrito, órdenes y usuario autenticado. Por ello, la capa L02 debe coordinar la conversación y delegar en servicios de aplicación, evitando que el proveedor de IA decida reglas de negocio o acceda directamente a la persistencia.

- Base bibliográfica: IEEE/ISO/IEC (2018) y Evans (2004).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.6 Arquitectura general del sistema

La arquitectura de software organiza las decisiones que permiten que un sistema cumpla requisitos funcionales y no funcionales. Sommerville (2016) relaciona arquitectura con rendimiento, seguridad, disponibilidad y mantenibilidad. En Aura Marketplace, la arquitectura general separa cliente, API, aplicación, dominio, infraestructura, datos e integraciones externas.

Esta separación permite explicar por qué el frontend no contiene secretos, por qué el backend centraliza reglas de negocio, por qué Prisma encapsula persistencia y por qué Mercado Pago, Resend, Cloudinary, Upstash Redis, Neon y Gemini deben tratarse como dependencias externas controladas. La arquitectura no solo representa el sistema; también limita cómo puede evolucionar.

- Base bibliográfica: Sommerville (2016) y Pressman y Maxim (2020).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.7 Arquitectura hexagonal

La arquitectura hexagonal, también conocida como puertos y adaptadores, propone proteger el núcleo de negocio de detalles externos. Cockburn y Garrido de Paz (2024) desarrollan este enfoque como una forma de simplificar la documentación técnica y separar el centro de la aplicación de sus mecanismos de entrada y salida. En términos de DDD, Evans (2004) refuerza que el dominio debe expresar el lenguaje y las reglas principales del negocio.

Aura Marketplace aplica esta teoría mediante L04 Domain, donde residen entidades, enums y puertos; L03 Application, donde se orquestan casos de uso; y L05 Infrastructure, donde se implementan adaptadores para Prisma, pagos, correo, cache, IA, almacenamiento y seguridad. Con ello, cambiar un proveedor no debería obligar a reescribir las reglas centrales.

- Base bibliográfica: Cockburn y Garrido de Paz (2024) y Evans (2004).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.8 Puertos, adaptadores e inversion de dependencias

Fowler (2004) analiza el patron conocido como Dependency Injection y lo relaciona con la inversion de control. En cita textual breve, Fowler aborda el tema bajo "the more specific name of Dependency Injection" (Fowler, 2004, par. 1). Este principio resulta esencial cuando el sistema necesita ensamblar implementaciones concretas sin acoplar la logica de aplicacion a una tecnologia puntual.

En Aura Marketplace, los puertos como IUserRepository, IOrderRepository, IPaymentGateway, IHasher, IMailSender o ICacheProvider definen contratos. Sus adaptadores concretos, como PrismaUserRepository, MercadoPagoService, Argon2HasherService o ResendMailService, satisfacen esos contratos. Esta decision permite pruebas con mocks, sustitucion de proveedores y menor riesgo de regresiones.

- Base bibliografica: Fowler (2004), Cockburn y Garrido de Paz (2024).
- Aplicacion en Aura Marketplace: la teoria se vincula con requisitos, diseño, implementacion y pruebas.
- Formato usado: citas parenteticas y narrativas segun APA 7.^a edicion.

2.9 Arquitectura del backend

NestJS se presenta en su documentación como un marco para construir aplicaciones del lado servidor eficientes y escalables. En cita textual breve, NestJS menciona "efficient, scalable Node.js server-side applications" (NestJS, s. f., par. 1). Esta definición justifica su elección para un backend modular que requiere controladores, servicios, guards, pipes y providers.

El backend de Aura Marketplace se divide en L01 Presentation, L02 Agent, L03 Application, L04 Domain y L05 Infrastructure. Esta estructura se alinea con la arquitectura hexagonal porque las capas externas reciben solicitudes o implementan detalles técnicos, mientras el dominio conserva contratos y conceptos del negocio. La modularidad de NestJS permite mapear requisitos a módulos funcionales.

- Base bibliográfica: NestJS (s. f.), Cockburn y Garrido de Paz (2024).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.10 Controladores, DTOs y validacion

La validacion de entradas es una practica fundamental de seguridad y calidad. OWASP Foundation (s. f.) propone usar controles tecnicos verificables para incrementar la confianza en aplicaciones web. En backend, los DTOs, pipes y validadores reducen la posibilidad de que datos invalidos lleguen a servicios de aplicacion o repositorios.

En Aura Marketplace, esta teoria se materializa en DTOs de registro, login, carrito, ordenes, categorias, perfil y restablecimiento de contrasena. La validacion tambien permite cumplir SDD: cada regla especificada debe tener una barrera tecnica que la haga exigible. Por ejemplo, una regla de precio mayor que cero debe aparecer en especificacion, DTO, servicio y prueba.

- Base bibliografica: OWASP Foundation (s. f.) y IEEE/ISO/IEC (2018).
- Aplicacion en Aura Marketplace: la teoria se vincula con requisitos, diseño, implementacion y pruebas.
- Formato usado: citas parenteticas y narrativas segun APA 7.^a edicion.

2.11 Seguridad backend

La seguridad por diseño exige controles en autenticación, gestión de sesiones, autorización, protección de datos, errores y auditoría. OWASP Foundation (s. f.) indica que ASVS ofrece una base para probar controles técnicos de seguridad. Esa orientación se alinea con sistemas que gestionan usuarios, pagos, roles y datos personales.

En Aura Marketplace, la seguridad se implementa mediante Argon2 para contraseñas, JWT y refresh tokens para sesiones, TokenRevocado para invalidación, RolesGuard para autorización, HTTPS en despliegue y auditoría para eventos críticos. La teoría respalda que estos controles no sean accesorios, sino requisitos no funcionales trazables.

- Base bibliográfica: OWASP Foundation (s. f.) y Pressman y Maxim (2020).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.12 Persistencia relacional y Prisma ORM

Prisma ORM se describe como una herramienta que proporciona acceso seguro por tipos, migraciones y editor visual de datos. En cita textual breve, Prisma destaca "type-safe database access" y migraciones (Prisma, s. f., par. 1). Esta idea resulta relevante para proyectos TypeScript donde el modelo de datos debe ser consistente con el código.

En Aura Marketplace, Prisma define entidades como Usuario, Publicacion, Inventario, Carrito, Orden, Pago, Sesión, Conversación, Mensaje, Intención, Auditoría y TokenRevocado. El esquema opera como contrato persistente del dominio, mientras los repositorios Prisma implementan puertos para aislar consultas, relaciones y transacciones de la capa de aplicación.

- Base bibliográfica: Prisma (s. f.) y Evans (2004).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.13 Integridad, transacciones y consistencia de datos

La calidad de datos se relaciona con reglas de negocio verificables, restricciones de integridad y comportamiento transaccional. Sommerville (2016) destaca que las restricciones no funcionales condicionan la arquitectura y la implementación. En un marketplace, la consistencia es crítica porque una orden no puede registrarse de forma separada al stock o al pago.

Aura Marketplace requiere transacciones para crear orden, líneas, pago, decremento de inventario y limpieza de carrito. Esta necesidad se deriva de SDD: las reglas de negocio de stock, precio histórico y confirmación de compra deben mantenerse aunque existan errores externos. Por eso la persistencia se concentra en adaptadores controlados.

- Base bibliográfica: Sommerville (2016), IEEE/ISO/IEC (2018).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.14 Arquitectura del frontend

React se presenta oficialmente como biblioteca para interfaces web y nativas. En cita textual breve, su documentación la define como "library for web and native user interfaces" (Meta Open Source, s. f., par. 1). Esta base teórica justifica su uso en una SPA donde las pantallas se componen desde unidades reutilizables.

En Aura Marketplace, el frontend organiza pages, components, api, store, layouts, hooks, lib, styles y types. Esta estructura soporta rutas públicas, rutas protegidas, panel del comprador, panel del vendedor y panel administrativo. El frontend no contiene reglas sensibles, pero sí debe representar estados, validar formularios y comunicar intenciones del usuario al backend.

- Base bibliográfica: Meta Open Source (s. f.) y Pressman y Maxim (2020).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.15 Componentes, reutilización y experiencia de usuario

El enfoque por componentes permite construir interfaces coherentes y mantenibles. Meta Open Source (s. f.) explica que React permite componer interfaces a partir de piezas individuales. En Aura Marketplace, esta teoría se refleja en componentes como AuraHeader, BrandLogo, ChatAgente, AuthLayout, AuthField, CheckoutStepper y CheckoutOrderSummary.

La reutilización no se limita a ahorrar código; también aporta consistencia visual y reduce errores. Cuando el sistema usa componentes para formularios de autenticación o checkout, las reglas de accesibilidad, estados de carga y mensajes de error pueden mantenerse de forma uniforme. Esto se vincula con requisitos de usabilidad y accesibilidad.

- Base bibliográfica: Meta Open Source (s. f.) y W3C Web Accessibility Initiative (s. f.).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.16 Estado cliente con Zustand

La gestión de estado cliente se vuelve necesaria cuando la interfaz debe recordar sesión, carrito, perfil, preferencias o conversación. Zustand se presenta como una solución pequeña, rápida y escalable de gestión de estado (Zustand, s. f.). En Aura Marketplace, esta característica se relaciona directamente con `authStore`, `cartStore`, `agentStore` y `profilePhotoStore`.

Desde SDD, cada store debe responder a una necesidad especificada: `authStore` sostiene autenticación, `cartStore` gestiona interacción de compra, `agentStore` conserva el estado conversacional y `profilePhotoStore` gestiona presentación del usuario. Así, el estado local no se convierte en lógica duplicada del backend, sino en soporte de experiencia.

- Base bibliográfica: Zustand (s. f.) y IEEE/ISO/IEC (2018).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.17 Estado servidor con TanStack Query

TanStack Query aborda el problema de sincronizar datos remotos en aplicaciones modernas. En cita textual breve, su documentacion resume: "Stop syncing server data by hand" (TanStack, s. f., par. 1). Esta idea se relaciona con cache, reintentos, invalidacion y actualizacion de datos remotos.

Aunque el proyecto tambien usa Axios y stores locales, el fundamento teorico de estado servidor permite justificar la separacion entre datos que pertenecen al backend y estados puramente visuales. Catalogo, ordenes, productos y reportes deben gestionarse como datos remotos, mientras modales, filtros visibles o mensajes temporales pueden ser estado local.

- Base bibliografica: TanStack (s. f.) y Meta Open Source (s. f.).
- Aplicacion en Aura Marketplace: la teoria se vincula con requisitos, diseño, implementacion y pruebas.
- Formato usado: citas parenteticas y narrativas segun APA 7.^a edicion.

2.18 Validación frontend y formularios

La validación en cliente mejora la experiencia de usuario al detectar errores antes de enviar solicitudes, pero no reemplaza la validación del backend. OWASP Foundation (s. f.) sostiene que los controles técnicos deben verificarse en la aplicación; por ello, la validación debe existir en varias capas. React Hook Form y Zod se entienden como soporte de frontera visual.

En Aura Marketplace, los formularios de registro, login, perfil, dirección, producto y checkout deben guiar al usuario con mensajes claros. Bajo SDD, cada campo requerido, restricción de contraseña o validación de precio debe estar especificada, implementada y probada. La validación cliente mejora usabilidad, mientras el backend asegura integridad.

- Base bibliográfica: OWASP Foundation (s. f.) y IEEE/ISO/IEC (2018).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.19 Accesibilidad web

La accesibilidad es una dimension de calidad que evita excluir usuarios por limitaciones visuales, motoras, cognitivas o tecnologicas. W3C Web Accessibility Initiative (s. f.) organiza WCAG bajo cuatro principios: "perceivable, operable, understandable, and robust" (par. 1). Esta cita textual breve sintetiza el marco de accesibilidad usado en aplicaciones web.

En Aura Marketplace, la accesibilidad se relaciona con navegacion por teclado, etiquetas en controles, mensajes de estado del agente, contraste visual, textos alternativos en imagenes y claridad de errores. El agente de voz tambien se justifica como mecanismo de acceso alternativo, pero no debe sustituir la compatibilidad general de la interfaz.

- Base bibliografica: W3C Web Accessibility Initiative (s. f.).
- Aplicacion en Aura Marketplace: la teoria se vincula con requisitos, diseño, implementacion y pruebas.
- Formato usado: citas parenteticas y narrativas segun APA 7.^a edicion.

2.20 Metodologías ágiles y Scrum

Scrum se define oficialmente como un marco para desarrollar y sostener productos complejos. En cita textual breve, Scrum Guides lo presenta como "framework for developing and sustaining complex products" (Schwaber & Sutherland, 2020, par. 1). Su valor teórico está en organizar trabajo empírico mediante roles, eventos, artefactos e incrementos.

En Aura Marketplace, Scrum permite agrupar el desarrollo por sprints: configuración base, autenticación, catálogo, carrito, pagos, agente, administración, calidad y despliegue. Al combinarse con SDD, el Product Backlog no nace de ideas sueltas, sino de especificaciones versionadas, criterios de aceptación y matriz de trazabilidad.

- Base bibliográfica: Schwaber y Sutherland (2020) y IEEE/ISO/IEC (2018).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.21 Product Backlog e historias de usuario

El Product Backlog representa el trabajo necesario para alcanzar el objetivo del producto. Schwaber y Sutherland (2020) explican que los artefactos de Scrum incrementan transparencia y permiten inspección. En este informe, las historias de usuario conectan necesidades del comprador, vendedor y administrador con funcionalidades verificables.

En Aura Marketplace, historias como registrarse, buscar productos, conversar con el agente, agregar al carrito, completar pago, publicar producto o administrar usuarios deben derivar de requisitos funcionales y reglas de negocio. Esta relación evita que el backlog pierda alineación con el documento SDD.

- Base bibliográfica: Schwaber y Sutherland (2020) y Pressman y Maxim (2020).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.22 Pruebas y aseguramiento de calidad

El aseguramiento de calidad no se reduce a ejecutar pruebas al final. Pressman y Maxim (2020) relacionan calidad con procesos, revisiones, mediciones y pruebas a lo largo del desarrollo. En un sistema modular, las pruebas deben cubrir unidades, integraciones y flujos E2E.

Aura Marketplace cuenta con pruebas unitarias en backend, pruebas de integración para proveedores y pruebas E2E en frontend. Bajo SDD, cada criterio de aceptación debe transformarse en evidencia de prueba. Por ejemplo, una regla de bloqueo por intentos fallidos debe aparecer en requisito, servicio, test y documentación.

- Base bibliográfica: Pressman y Maxim (2020) y IEEE/ISO/IEC (2018).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.23 Seguridad aplicativa y ASVS

OWASP ASVS ofrece un marco para verificar controles técnicos de seguridad. En cita textual breve, ASVS proporciona una "basis for testing application technical security controls" (OWASP Foundation, s. f., par. 1). Esto permite transformar seguridad en requisitos verificables y no en una declaración general.

En Aura Marketplace, ASVS respalda decisiones como hash de contraseñas, expiración de tokens, control de acceso por roles, validación de entrada, no almacenamiento de tarjetas, auditoría y manejo seguro de errores. La seguridad se documenta como parte del diseño y se prueba como parte del ciclo de calidad.

- Base bibliográfica: OWASP Foundation (s. f.).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.24 Despliegue y entrega continua

La entrega continua promueve que el software pueda construirse, probarse y desplegarse de forma repetible. Aunque Aura Marketplace no necesariamente automatiza todo el ciclo, su documentación de Render, Vercel, Prisma Migrate, variables de entorno y GitHub Actions se alinea con prácticas de despliegue controlado (Humble & Farley, 2010).

Desde SDD, el despliegue también debe estar especificado: variables obligatorias, comandos de build, migraciones, dominios, CORS, webhooks y health checks. Esto reduce errores entre ambientes y mantiene coherencia entre la arquitectura diseñada y la operación real.

- Base bibliográfica: Humble y Farley (2010) y Pressman y Maxim (2020).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.25 Integraciones externas

Las integraciones externas deben tratarse como dependencias que pueden fallar, cambiar o introducir latencia. Sommerville (2016) sostiene que los requisitos no funcionales condicionan decisiones arquitectónicas. Por ello, servicios como Mercado Pago, Resend, Cloudinary, Upstash Redis, Neon Auth y Gemini AI deben encapsularse mediante adaptadores.

En Aura Marketplace, este principio permite degradación controlada. Si falla correo, el sistema debe informar; si falla IA, puede mantenerse navegación manual; si falla pago, el carrito no debe perderse. La teoría respalda que cada integración tenga contrato, manejo de errores, configuración segura y pruebas.

- Base bibliográfica: Sommerville (2016), Cockburn y Garrido de Paz (2024).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.26 Observabilidad y auditoria

La observabilidad permite comprender el comportamiento del sistema mediante eventos, logs, metricas y trazas. Pressman y Maxim (2020) asocian la calidad con capacidad de diagnostico y mantenimiento. En sistemas transaccionales, la auditoria tambien cumple una funcion de confianza y responsabilidad.

Aura Marketplace registra eventos relevantes como autenticaciones, instrucciones del agente, cambios de publicaciones, registro de ordenes, cambios de estado y errores de integraciones. La auditoria debe evitar datos sensibles, cumpliendo principios de seguridad. Bajo SDD, cada evento auditable debe relacionarse con requisito o regla de negocio.

- Base bibliografica: Pressman y Maxim (2020) y OWASP Foundation (s. f.).
- Aplicacion en Aura Marketplace: la teoria se vincula con requisitos, diseño, implementacion y pruebas.
- Formato usado: citas parenteticas y narrativas segun APA 7.^a edicion.

2.27 Dominio, entidades y lenguaje ubicuo

Domain-Driven Design propone que el lenguaje del dominio guíe el modelo del software. Evans (2004) sostiene que el centro del diseño debe expresar conceptos y reglas relevantes para el negocio. En Aura Marketplace, términos como Usuario, Publicación, Orden, Carrito, Pago, Sesión, Mensaje e Intención deben usarse de forma consistente.

El glosario de specs cumple una función compatible con DDD: evita que una misma idea tenga varios nombres y permite alinear base de datos, DTOs, servicios, stores y documentación. Esto mejora mantenibilidad porque los desarrolladores pueden entender el sistema desde el lenguaje del negocio.

- Base bibliográfica: Evans (2004) y IEEE/ISO/IEC (2018).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.28 Modularidad y mantenibilidad

La mantenibilidad se favorece cuando los módulos tienen responsabilidades claras, interfaces estables y dependencias controladas. Sommerville (2016) y Pressman y Maxim (2020) coinciden en que la estructura del software afecta el costo de cambio. En Aura Marketplace, la separación de módulos permite evolucionar autenticación, catálogo, pagos o agente sin reescribir todo el sistema.

La modularidad también facilita pruebas. Un servicio puede probarse con repositorios simulados, un adaptador puede verificarse con pruebas de integración y una página frontend puede evaluarse mediante flujos E2E. El marco teórico respalda que la arquitectura documentada sea parte de la calidad del producto.

- Base bibliográfica: Sommerville (2016) y Pressman y Maxim (2020).
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

2.29 Relacion entre SDD, arquitectura y Scrum

SDD define que se debe construir; arquitectura define como se organiza; Scrum define como se gestiona el trabajo iterativo. Estas tres perspectivas no compiten: se complementan. IEEE/ISO/IEC (2018) aporta el enfoque de requisitos, Cockburn y Garrido de Paz (2024) aportan separacion arquitectonica y Schwaber y Sutherland (2020) aportan inspeccion y adaptacion.

En Aura Marketplace, esta relacion permite que cada sprint tenga una base documental, cada modulo tenga una responsabilidad tecnica y cada prueba responda a un criterio de aceptacion. El informe, por tanto, no solo describe el resultado final; tambien muestra el camino metodologico y tecnico para justificarlo.

- Base bibliografica: IEEE/ISO/IEC (2018), Cockburn y Garrido de Paz (2024), Schwaber y Sutherland (2020).
- Aplicacion en Aura Marketplace: la teoria se vincula con requisitos, diseño, implementacion y pruebas.
- Formato usado: citas parenteticas y narrativas segun APA 7.^a edicion.

2.30 Síntesis del marco teórico

El marco teórico demuestra que Aura Marketplace se sostiene en fundamentos de ingeniería de software, requisitos, arquitectura, frontend, backend, seguridad, accesibilidad, persistencia, pruebas y metodologías ágiles. La principal conclusión teórica es que el sistema no debe entenderse como una suma de tecnologías, sino como un producto guiado por especificaciones y validado por evidencias.

Por ello, el informe adopta SDD como eje documental. Las referencias bibliográficas permiten sustentar que cada decisión tiene respaldo teórico: React para componentes, NestJS para backend modular, Prisma para persistencia tipada, Scrum para gestión iterativa, ASVS para seguridad, WCAG para accesibilidad y arquitectura hexagonal para desacoplamiento.

- Base bibliográfica: todas las fuentes citadas en el Capítulo II.
- Aplicación en Aura Marketplace: la teoría se vincula con requisitos, diseño, implementación y pruebas.
- Formato usado: citas parentéticas y narrativas según APA 7.^a edición.

Capítulo III

3.1 Tipo de investigación

En el contexto de Aura Marketplace, una investigación aplicada-tecnológica orientada a resolver un problema concreto mediante un producto de software documentado se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodología, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revisión o retrospectiva.
- Evidencia esperada: código, prueba, tabla, diagrama o criterio de aceptación.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.2 Nivel de investigación

En el contexto de Aura Marketplace, un nivel descriptivo porque identifica componentes, procesos, tecnologías, actores y flujos sin manipular variables experimentales se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodología, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revisión o retrospectiva.
- Evidencia esperada: código, prueba, tabla, diagrama o criterio de aceptación.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.3 Diseño de investigación

En el contexto de Aura Marketplace, un diseño no experimental y transversal aplicado al análisis del repositorio, los diagramas y los documentos de especificación se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodología, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revisión o retrospectiva.
- Evidencia esperada: código, prueba, tabla, diagrama o criterio de aceptación.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.4 Poblacion y muestra tecnica

En el contexto de Aura Marketplace, los actores del sistema, modulos del repositorio, requisitos, casos de uso, entidades de datos y diagramas de arquitectura se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodologia, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revision o retrospectiva.
- Evidencia esperada: codigo, prueba, tabla, diagrama o criterio de aceptacion.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.5 Variables e indicadores

En el contexto de Aura Marketplace, funcionamiento, usabilidad técnica, mantenibilidad, seguridad, escalabilidad, trazabilidad y cobertura documental se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodología, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revisión o retrospectiva.
- Evidencia esperada: código, prueba, tabla, diagrama o criterio de aceptación.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.6 Tecnicas de recoleccion

En el contexto de Aura Marketplace, revision documental, inspeccion de codigo, analisis de diagramas, lectura del PDF de referencia y construccion de anexos se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodologia, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revision o retrospectiva.
- Evidencia esperada: codigo, prueba, tabla, diagrama o criterio de aceptacion.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.7 Instrumentos

En el contexto de Aura Marketplace, fichas de análisis documental, matriz de consistencia, backlog, matriz de trazabilidad, checklist de arquitectura y escenarios SDD se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodología, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revisión o retrospectiva.
- Evidencia esperada: código, prueba, tabla, diagrama o criterio de aceptación.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.8 Metodo Scrum

En el contexto de Aura Marketplace, organizacion iterativa en sprints con backlog, sprint planning, daily review, incrementos y retrospectiva se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodologia, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revision o retrospectiva.
- Evidencia esperada: codigo, prueba, tabla, diagrama o criterio de aceptacion.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.9 Roles Scrum

En el contexto de Aura Marketplace, Product Owner, Scrum Master, Development Team y stakeholders académicos del proyecto se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodología, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revisión o retrospectiva.
- Evidencia esperada: código, prueba, tabla, diagrama o criterio de aceptación.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.10 Product Backlog

En el contexto de Aura Marketplace, historias de usuario para agente, autenticación, catálogo, carrito, checkout, vendedor, administrador, notificaciones y despliegue se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodología, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revisión o retrospectiva.
- Evidencia esperada: código, prueba, tabla, diagrama o criterio de aceptación.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.11 Sprint Planning

En el contexto de Aura Marketplace, selección de historias por prioridad, complejidad, dependencias técnicas y valor de negocio se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodología, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revisión o retrospectiva.
- Evidencia esperada: código, prueba, tabla, diagrama o criterio de aceptación.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.12 Sprint 1

En el contexto de Aura Marketplace, preparación del monorepo, configuración base, estructura de carpetas, validaciones iniciales y entorno de desarrollo se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodología, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revisión o retrospectiva.
- Evidencia esperada: código, prueba, tabla, diagrama o criterio de aceptación.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.13 Sprint 2

En el contexto de Aura Marketplace, autenticación, usuarios, roles, JWT, refresh tokens, Neon Auth y verificación de correo se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodología, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revisión o retrospectiva.
- Evidencia esperada: código, prueba, tabla, diagrama o criterio de aceptación.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.14 Sprint 3

En el contexto de Aura Marketplace, catalogo, categorias, publicaciones, inventario, imagenes y busqueda se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodologia, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revision o retrospectiva.
- Evidencia esperada: codigo, prueba, tabla, diagrama o criterio de aceptacion.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.15 Sprint 4

En el contexto de Aura Marketplace, carrito, ordenes, pagos, Mercado Pago y consistencia transaccional se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodología, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revisión o retrospectiva.
- Evidencia esperada: código, prueba, tabla, diagrama o criterio de aceptación.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.16 Sprint 5

En el contexto de Aura Marketplace, agente inteligente, interpretación de lenguaje natural, audio, conversaciones, intenciones y entidades se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodología, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revisión o retrospectiva.
- Evidencia esperada: código, prueba, tabla, diagrama o criterio de aceptación.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.17 Sprint 6

En el contexto de Aura Marketplace, administracion, reportes, auditoria, favoritos, resenas, promociones y notificaciones se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodologia, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por disenio y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se disenaa para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revision o retrospectiva.
- Evidencia esperada: codigo, prueba, tabla, diagrama o criterio de aceptacion.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.18 Sprint 7

En el contexto de Aura Marketplace, despliegue, CI/CD, Render, Vercel, variables de entorno, pruebas e integraciones se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodología, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revisión o retrospectiva.
- Evidencia esperada: código, prueba, tabla, diagrama o criterio de aceptación.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.19 Definición de terminado

En el contexto de Aura Marketplace, código compilable, pruebas relevantes, endpoints documentados, validaciones, seguridad y evidencia en anexos se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodología, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revisión o retrospectiva.
- Evidencia esperada: código, prueba, tabla, diagrama o criterio de aceptación.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

3.20 Gestion de riesgos

En el contexto de Aura Marketplace, fallos de proveedores externos, latencia, credenciales, consistencia de datos, acoplamiento y regresiones de frontend se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de metodologia, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseno y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se disena para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Artefacto Scrum asociado: backlog, sprint, incremento, revision o retrospectiva.
- Evidencia esperada: codigo, prueba, tabla, diagrama o criterio de aceptacion.
- Criterio de control: trazabilidad con requisitos funcionales y no funcionales.

Capítulo IV

4.1 Vision del producto implementado

En el contexto de Aura Marketplace, Aura Marketplace como plataforma multidireccional de comercio electrónico con agente conversacional se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.2 Actores del sistema

En el contexto de Aura Marketplace, visitante, comprador, vendedor, administrador y servicios externos se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.3 Modulos funcionales

En el contexto de Aura Marketplace, autenticacion, usuarios, catalogo, carrito, ordenes, pagos, agente, administracion, auditoria y notificaciones se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por disenio y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se disenia para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, disenio, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.4 Arquitectura general del sistema

En el contexto de Aura Marketplace, la relación entre frontend React, backend NestJS, base de datos, integraciones y despliegue se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.5 Flujo cliente-servidor

En el contexto de Aura Marketplace, la comunicación HTTPS REST entre navegador, API, capa de aplicación, infraestructura y datos se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.6 Arquitectura del backend L01

En el contexto de Aura Marketplace, controladores REST, guards, DTOs, ValidationPipe, CORS, Swagger y endpoints se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.7 Arquitectura del backend L02

En el contexto de Aura Marketplace, AgentService y ConversationsService como capa de inteligencia conversacional se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.8 Arquitectura del backend L03

En el contexto de Aura Marketplace, servicios de aplicación y casos de uso para productos, carrito, ordenes, pagos, usuarios y admin se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.9 Arquitectura del backend L04

En el contexto de Aura Marketplace, entidades, enums, puertos de repositorio y contratos de proveedores se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.10 Arquitectura del backend L05

En el contexto de Aura Marketplace, adaptadores Prisma, Cloudinary, Mercado Pago, Resend, Argon2, Redis y Gemini se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.11 Inyección de dependencias

En el contexto de Aura Marketplace, tokens de interfaces y providers para desacoplar servicios de aplicación de implementaciones concretas se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.12 Controladores REST

En el contexto de Aura Marketplace, admin, agent, auth, cart, categories, favorites, notifications, orders, payments, products, promotions, reviews y users se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.13 Guards y autorizacion

En el contexto de Aura Marketplace, JwtAuthGuard, RolesGuard, decoradores publicos y decoradores de roles se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.14 DTOs y validaciones

En el contexto de Aura Marketplace, login, registro, refresh, verify email, password reset, carrito, ordenes, categorias y perfiles se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.15 Servicios de aplicación

En el contexto de Aura Marketplace, AuthService, OrdersService, ProductsService, CartService, PaymentsService, UsersService y AdminService se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.16 Repositorios Prisma

En el contexto de Aura Marketplace, adaptadores por entidad para usuario, producto, orden, carrito, auditoria, conversaciones y notificaciones se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseno y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se disena para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, diseno, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.17 Transacciones de orden

En el contexto de Aura Marketplace, creación atómica de orden, líneas, pago, decremento de inventario y limpieza de carrito se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.18 Seguridad de contraseñas

En el contexto de Aura Marketplace, Argon2 como derivación segura y separación del hasher mediante puerto se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.19 Gestion de tokens

En el contexto de Aura Marketplace, JWT access token, refresh token, TokenRevocado y bloqueo por intentos fallidos se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por disenio y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se disenia para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, disenio, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.20 Neon Auth

En el contexto de Aura Marketplace, sincronización de identidad externa con usuarios locales del sistema se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.21 Mercado Pago

En el contexto de Aura Marketplace, integración de checkout, referencia de pago, webhook y estados de pago se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.22 Resend Email API

En el contexto de Aura Marketplace, verificación de cuenta, restablecimiento de contraseña y comunicaciones transaccionales se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.23 Cloudinary

En el contexto de Aura Marketplace, almacenamiento de imágenes de producto y separación de assets del backend se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.24 Upstash Redis

En el contexto de Aura Marketplace, cache distribuida para reducir latencia y preservar escalabilidad se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por disenio y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se disenaa para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, disenio, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.25 Gemini AI

En el contexto de Aura Marketplace, procesamiento de lenguaje natural e interpretación de intenciones del agente se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.26 Arquitectura del frontend

En el contexto de Aura Marketplace, React modular por pages, components, api, store, layouts, hooks, lib, styles y types se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.27 Enrutamiento frontend

En el contexto de Aura Marketplace, `BrowserRouter`, rutas publicas, rutas protegidas, perfiles y panel de administracion se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.28 Paginas publicas

En el contexto de Aura Marketplace, Home, Catalog, ProductDetail, Login, Register, ForgotPassword, ResetPassword y VerifyEmail se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por disenio y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se disenia para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, disenio, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.29 Flujos de comprador

En el contexto de Aura Marketplace, carrito, checkout shipping, checkout payment, order success, favoritos e historial de ordenes se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.30 Flujos de vendedor

En el contexto de Aura Marketplace, productos del vendedor, formulario de producto, ordenes del vendedor y clientes se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por disenio y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se disenia para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, disenio, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.31 Flujos de administrador

En el contexto de Aura Marketplace, usuarios, productos, ordenes, categorias, reportes y perfil de administracion se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.32 Componentes visuales

En el contexto de Aura Marketplace, AuraHeader, BrandLogo, ChatAgente, AuthLayout, AuthField, CheckoutStepper y componentes UI se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.33 Estado global frontend

En el contexto de Aura Marketplace, authStore, cartStore, agentStore y profilePhotoStore se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.34 Cliente HTTP

En el contexto de Aura Marketplace, axios, interceptores, VITE_API_URL, tokens y manejo de errores se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.35 Validación cliente

En el contexto de Aura Marketplace, Zod y React Hook Form como filtro de calidad antes de la API se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.36 Diseño visual

En el contexto de Aura Marketplace, Tailwind CSS, shadcn/ui, componentes reutilizables y consistencia de layouts se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.37 Base de datos

En el contexto de Aura Marketplace, modelo relacional en Prisma con entidades de identidad, catalogo, marketplace, agente y auditoria se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por disenio y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se disenya para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, disenio, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.38 Entidades de identidad

En el contexto de Aura Marketplace, Usuario, RefreshToken, Direccion, PreferenciasUsuario y TokenRevocado se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.39 Entidades de catalogo

En el contexto de Aura Marketplace, Categoría, Publicación, Inventario e ImagenPublicación se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.40 Entidades transaccionales

En el contexto de Aura Marketplace, Carrito, ItemCarrito, Orden, LineaOrden, Pago, Cupon y Promocion se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.41 Entidades del agente

En el contexto de Aura Marketplace, Sesión, Conversación, Mensaje, Intención y EntidadExtraída se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.42 Entidades de soporte

En el contexto de Aura Marketplace, Favorito, Resena, Notificación y Auditoria se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.43 Integridad referencial

En el contexto de Aura Marketplace, claves foraneas, relaciones en cascada, índices y unicidad se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.44 Reglas de negocio

En el contexto de Aura Marketplace, confirmación de compra, stock atómico, precio positivo, validación de datos y auditoría se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.45 Requisitos no funcionales

En el contexto de Aura Marketplace, rendimiento, disponibilidad, seguridad, escalabilidad, usabilidad, accesibilidad y mantenibilidad se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.46 Pruebas unitarias backend

En el contexto de Aura Marketplace, specs por controladores, servicios, repositorios, providers y utilidades se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.47 Pruebas de integracion

En el contexto de Aura Marketplace, auth, RBAC, pagos, ordenes, base de datos, Redis, Cloudinary, Resend, Gemini y agente se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.48 Pruebas E2E frontend

En el contexto de Aura Marketplace, flujos de compra, vendedor, administrador, autenticación y agente se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.49 Despliegue backend

En el contexto de Aura Marketplace, Render, build NestJS, Prisma generate, migrate deploy y health check se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.50 Despliegue frontend

En el contexto de Aura Marketplace, Vercel o Pages, build Vite, dist y rewrites SPA se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.51 Variables de entorno

En el contexto de Aura Marketplace, separación de secretos, URLs públicas, claves de proveedores y credenciales de despliegue se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.52 CI/CD

En el contexto de Aura Marketplace, GitHub Actions para validar frontend, backend, Prisma, tests y disparar despliegues se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.53 Observabilidad

En el contexto de Aura Marketplace, logs, auditoria, health checks y trazabilidad de eventos criticos se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por disenio y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se disenaa para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, disenio, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.54 Discusion de resultados

En el contexto de Aura Marketplace, alineacion entre teoria, arquitectura, implementacion y anexos se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por disenio y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se disenaa para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Fuente tecnica: repositorio, especificaciones, disenio, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: modulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

4.55 Síntesis técnica

En el contexto de Aura Marketplace, madurez del diseño, pendientes de pruebas de carga y mejora continua se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de resultados, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Fuente técnica: repositorio, especificaciones, diseño, schema Prisma o diagrama anexo.
- Impacto: mejora de mantenibilidad, seguridad, trazabilidad o experiencia del usuario.
- Evidencia: módulo, entidad, flujo, servicio, adaptador, prueba o variable de entorno.

Capítulo V

5.1 Conclusion general

En el contexto de Aura Marketplace, Aura Marketplace evidencia una arquitectura coherente para un marketplace inteligente porque separa la experiencia de usuario, la lógica de aplicación, el dominio, la infraestructura y los servicios externos. Se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de conclusiones, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Mantener la documentación sincronizada con el repositorio.
- Usar anexos como evidencia de diseño y no como material decorativo.
- Registrar decisiones arquitectónicas cuando cambien proveedores o reglas de negocio.

5.2 Conclusion sobre SDD

En el contexto de Aura Marketplace, Specs Driven Development permite que las especificaciones actúen como fuente de verdad para requisitos, diseño, implementación, pruebas y anexos académicos. Se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de conclusiones, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Mantener la documentación sincronizada con el repositorio.
- Usar anexos como evidencia de diseño y no como material decorativo.
- Registrar decisiones arquitectónicas cuando cambien proveedores o reglas de negocio.

5.3 Conclusion sobre backend

En el contexto de Aura Marketplace, la estructura NestJS L01-L05 fortalece la mantenibilidad al ubicar controladores, agente, casos de uso, puertos y adaptadores en límites identificables. Se entiende como un componente crítico del ciclo de vida del sistema, porque permite conectar las decisiones técnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnología; también explica por qué esa tecnología se adopta y qué consecuencias tiene sobre la calidad final.

Desde la perspectiva de conclusiones, el proyecto requiere separar los conceptos teóricos de las decisiones de implementación. Esta separación evita que el documento sea una simple descripción de pantallas o archivos y permite demostrar que cada módulo responde a principios de ingeniería: bajo acoplamiento, alta cohesión, trazabilidad, seguridad por diseño y validación continua.

La aplicación concreta se observa en la integración entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el núcleo del dominio.

- Mantener la documentación sincronizada con el repositorio.
- Usar anexos como evidencia de diseño y no como material decorativo.
- Registrar decisiones arquitectónicas cuando cambien proveedores o reglas de negocio.

5.4 Conclusion sobre frontend

En el contexto de Aura Marketplace, la SPA React organiza rutas, componentes, stores y clientes API de forma modular, permitiendo que compradores, vendedores y administradores trabajen con flujos diferenciados. se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de conclusiones, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por disenio y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se disenaa para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Mantener la documentacion sincronizada con el repositorio.
- Usar anexos como evidencia de disenio y no como material decorativo.
- Registrar decisiones arquitectonicas cuando cambien proveedores o reglas de negocio.

5.5 Conclusion sobre metodología

En el contexto de Aura Marketplace, Scrum proporciona una ruta iterativa para planificar, construir, revisar y estabilizar la plataforma por incrementos verificables. se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de conclusiones, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por disenio y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se disenia para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Mantener la documentacion sincronizada con el repositorio.
- Usar anexos como evidencia de disenio y no como material decorativo.
- Registrar decisiones arquitectonicas cuando cambien proveedores o reglas de negocio.

5.6 Recomendaciones

En el contexto de Aura Marketplace, se recomienda ejecutar pruebas de carga, auditoria de accesibilidad, monitoreo en produccion, revision de secretos, actualizacion de diagramas y mantenimiento de backlog. se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de conclusiones, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por disenio y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se disenaa para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Mantener la documentacion sincronizada con el repositorio.
- Usar anexos como evidencia de disenio y no como material decorativo.
- Registrar decisiones arquitectonicas cuando cambien proveedores o reglas de negocio.

Referencias bibliograficas

Las siguientes referencias se presentan en formato APA 7.^a edición y sustentan principalmente el Capítulo II, donde se desarrolla el marco teórico de SDD, arquitectura, backend, frontend, seguridad, accesibilidad, persistencia, pruebas y Scrum.

Cockburn, A., & Garrido de Paz, J. M. (2024). *Hexagonal Architecture Explained: How the Ports & Adapters Architecture Simplifies Your Technical Documentation*. Humans and Technology.

Evans, E. (2004). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

Fowler, M. (2004). *Inversion of Control Containers and the Dependency Injection Pattern*. <https://martinfowler.com/articles/injection.html>

Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.

IEEE/ISO/IEC (2018). *IEEE/ISO/IEC 29148-2018: Systems and software engineering - Life cycle processes - Requirements engineering*. IEEE Standards Association. <https://standards.ieee.org/standard/29148-2018.html>

Meta Open Source (s. f.). *React*. <https://react.dev/>

NestJS (s. f.). *Documentation*. <https://docs.nestjs.com/>

OWASP Foundation (s. f.). *OWASP Application Security Verification Standard (ASVS)*. <https://owasp.org/www-project-application-security-verification-standard/>

Prisma (s. f.). *What is Prisma ORM?* <https://www.prisma.io/docs/orm>

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach (9th ed.)*. McGraw-Hill Education.

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. <https://scrumguides.org/>

Sommerville, I. (2016). *Software Engineering (10th ed.)*. Pearson.

TanStack (s. f.). *TanStack Query*. <https://tanstack.com/query/latest>

W3C Web Accessibility Initiative (s. f.). *WCAG 2 Overview*.

<https://www.w3.org/WAI/standards-guidelines/wcag/>

Zustand (s. f.). *Introduction*. <https://zustand.docs.pmnd.rs/>

Anexos

Anexo 1. Matriz de consistencia

La matriz de consistencia resume la relación entre problema, objetivos, hipótesis, variables e instrumentos usados para elaborar el informe.

Elemento	Descripción
Problema	Fricción en comercio electrónico y complejidad técnica de una plataforma inteligente.
Objetivo	Documentar arquitectura, backend, frontend, Scrum, SDD y anexos de Aura Marketplace.
Hipótesis	Una documentación basada en SDD y arquitectura hexagonal mejora trazabilidad y mantenibilidad.
Variables	Funcionamiento, mantenibilidad, seguridad, usabilidad técnica y trazabilidad.
Instrumentos	Revisión documental, inspección de código, matriz de requisitos, backlog y diagramas.

Anexo 2. Product Backlog

El backlog representa el conjunto inicial de historias necesarias para construir la experiencia principal del marketplace y del agente inteligente.

ID	Historia	Prioridad	Puntos
HU-01	Registrarse y verificar correo	Alta	5
HU-02	Iniciar sesion y renovar token	Alta	5
HU-03	Buscar productos con filtros	Alta	8
HU-04	Conversar con el agente	Alta	13
HU-05	Agregar producto al carrito	Alta	5
HU-06	Completar checkout y pago	Alta	13
HU-07	Publicar producto como vendedor	Alta	8
HU-08	Gestionar ordenes de vendedor	Media	8
HU-09	Administrar usuarios y productos	Alta	8
HU-10	Consultar reportes y auditoria	Media	5
HU-11	Recibir correos transaccionales	Media	5
HU-12	Subir imagenes de publicaciones	Media	5

Anexo 3. SDD-01 Vision del producto

Este anexo explica como se aplica SDD en el proyecto: la especificacion funciona como fuente de verdad antes de disenar, codificar o probar.

La aplicacion de SDD en el informe permite conectar requisitos, reglas de negocio, casos de uso, disenar arquitectonico, implementacion y pruebas de aceptacion.

Paso	Elemento SDD
1	Spec de origen: specs/01-Vision.md
2	Define problema, oportunidad, propuesta de valor y riesgos
3	Guia: objetivos estrategicos, arquitectura inicial y alcance del MVP

Anexo 3. SDD-02 Stakeholders y actores

Este anexo explica como se aplica SDD en el proyecto: la especificacion funciona como fuente de verdad antes de disenar, codificar o probar.

La aplicacion de SDD en el informe permite conectar requisitos, reglas de negocio, casos de uso, disenar arquitectonico, implementacion y pruebas de aceptacion.

Paso	Elemento SDD
1	Spec de origen: specs/02-Stakeholders.md y specs/09-Actores.md
2	Define comprador, vendedor, administrador y servicios externos
3	Guia: permisos, rutas protegidas y experiencia por rol

Anexo 3. SDD-03 Glosario del dominio

Este anexo explica como se aplica SDD en el proyecto: la especificacion funciona como fuente de verdad antes de disenar, codificar o probar.

La aplicacion de SDD en el informe permite conectar requisitos, reglas de negocio, casos de uso, disenar arquitectonico, implementacion y pruebas de aceptacion.

Paso	Elemento SDD
1	Spec de origen: specs/03-Glosario.md
2	Normaliza terminos como Usuario, Publicacion, Orden, Carrito, Agente e Intencion
3	Guia: nombres de entidades, DTOs, stores y tablas

Anexo 3. SDD-04 Alcance

Este anexo explica como se aplica SDD en el proyecto: la especificacion funciona como fuente de verdad antes de disenar, codificar o probar.

La aplicacion de SDD en el informe permite conectar requisitos, reglas de negocio, casos de uso, disenar arquitectonico, implementacion y pruebas de aceptacion.

Paso	Elemento SDD
1	Spec de origen: specs/04-Alcance.md
2	Separa lo implementado de lo fuera de alcance
3	Guia: priorizacion del backlog y control de expectativas

Anexo 3. SDD-05 Objetivos

Este anexo explica como se aplica SDD en el proyecto: la especificacion funciona como fuente de verdad antes de disenar, codificar o probar.

La aplicacion de SDD en el informe permite conectar requisitos, reglas de negocio, casos de uso, disenio arquitectonico, implementacion y pruebas de aceptacion.

Paso	Elemento SDD
1	Spec de origen: specs/05-Objetivos.md
2	Define objetivos e indicadores medibles
3	Guia: KPIs, pruebas y criterios de aceptacion

Anexo 3. SDD-06 Reglas de negocio

Este anexo explica como se aplica SDD en el proyecto: la especificacion funciona como fuente de verdad antes de disenar, codificar o probar.

La aplicacion de SDD en el informe permite conectar requisitos, reglas de negocio, casos de uso, disenar arquitectonico, implementacion y pruebas de aceptacion.

Paso	Elemento SDD
1	Spec de origen: specs/06-ReglasNegocio.md
2	Define restricciones como stock, precio, confirmacion, bloqueo y auditoria
3	Guia: servicios de aplicacion, validaciones y transacciones

Anexo 3. SDD-07 Requisitos funcionales

Este anexo explica como se aplica SDD en el proyecto: la especificacion funciona como fuente de verdad antes de disenar, codificar o probar.

La aplicacion de SDD en el informe permite conectar requisitos, reglas de negocio, casos de uso, disenar arquitectonico, implementacion y pruebas de aceptacion.

Paso	Elemento SDD
1	Spec de origen: specs/07-RequisitosFuncionales.md
2	Describe funciones por modulo y actor
3	Guia: controladores REST, paginas, componentes y casos de prueba

Anexo 3. SDD-08 Requisitos no funcionales

Este anexo explica como se aplica SDD en el proyecto: la especificacion funciona como fuente de verdad antes de disenar, codificar o probar.

La aplicacion de SDD en el informe permite conectar requisitos, reglas de negocio, casos de uso, disenar arquitectonico, implementacion y pruebas de aceptacion.

Paso	Elemento SDD
1	Spec de origen: specs/08-RequisitosNoFuncionales.md
2	Define rendimiento, disponibilidad, seguridad, escalabilidad y accesibilidad
3	Guia: arquitectura cloud, cache, seguridad y observabilidad

Anexo 3. SDD-09 Casos de uso y criterios

Este anexo explica como se aplica SDD en el proyecto: la especificacion funciona como fuente de verdad antes de disenar, codificar o probar.

La aplicacion de SDD en el informe permite conectar requisitos, reglas de negocio, casos de uso, disenar arquitectonico, implementacion y pruebas de aceptacion.

Paso	Elemento SDD
1	Spec de origen: specs/10-CasosUsoResumen.md y specs/11-CriteriosAceptacion.md
2	Concreta comportamiento esperado en escenarios verificables
3	Guia: pruebas unitarias, integracion y E2E

Anexo 3. SDD-10 Matriz de trazabilidad

Este anexo explica como se aplica SDD en el proyecto: la especificacion funciona como fuente de verdad antes de disenar, codificar o probar.

La aplicacion de SDD en el informe permite conectar requisitos, reglas de negocio, casos de uso, disenar arquitectonico, implementacion y pruebas de aceptacion.

Paso	Elemento SDD
1	Spec de origen: specs/12-MatrizTrazabilidad.md
2	Relaciona objetivos, requisitos, reglas y casos de uso
3	Guia: cierre documental, anexos y validacion academica

Anexo 4.1. Arquitectura general del backend y servicios externos

La figura se incorpora como evidencia visual del diseño proporcionado. Su lectura complementa las secciones de arquitectura general, backend, frontend, persistencia e integraciones.

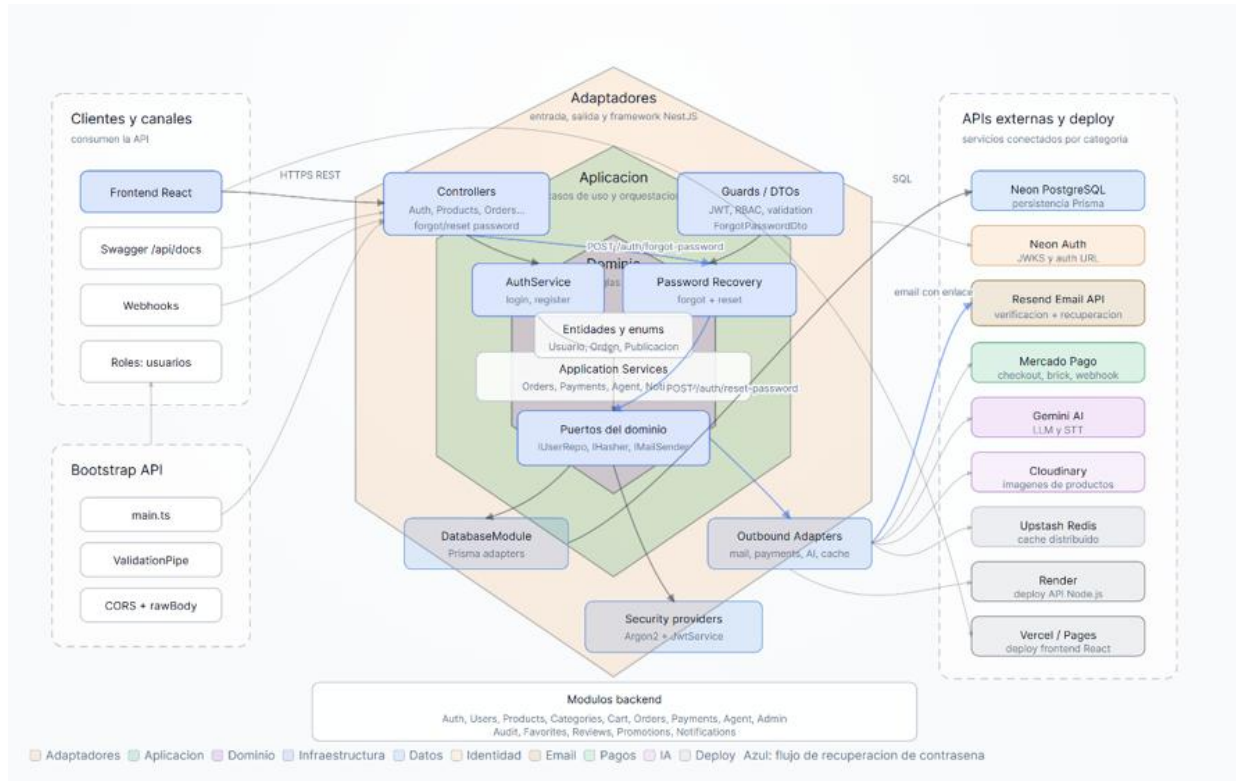


Figura A1. Arquitectura general del backend y servicios externos. Fuente: imagen proporcionada para anexos del proyecto.

Anexo 4.2. Arquitectura modular del frontend React

La figura se incorpora como evidencia visual del diseño proporcionado. Su lectura complementa las secciones de arquitectura general, backend, frontend, persistencia e integraciones.

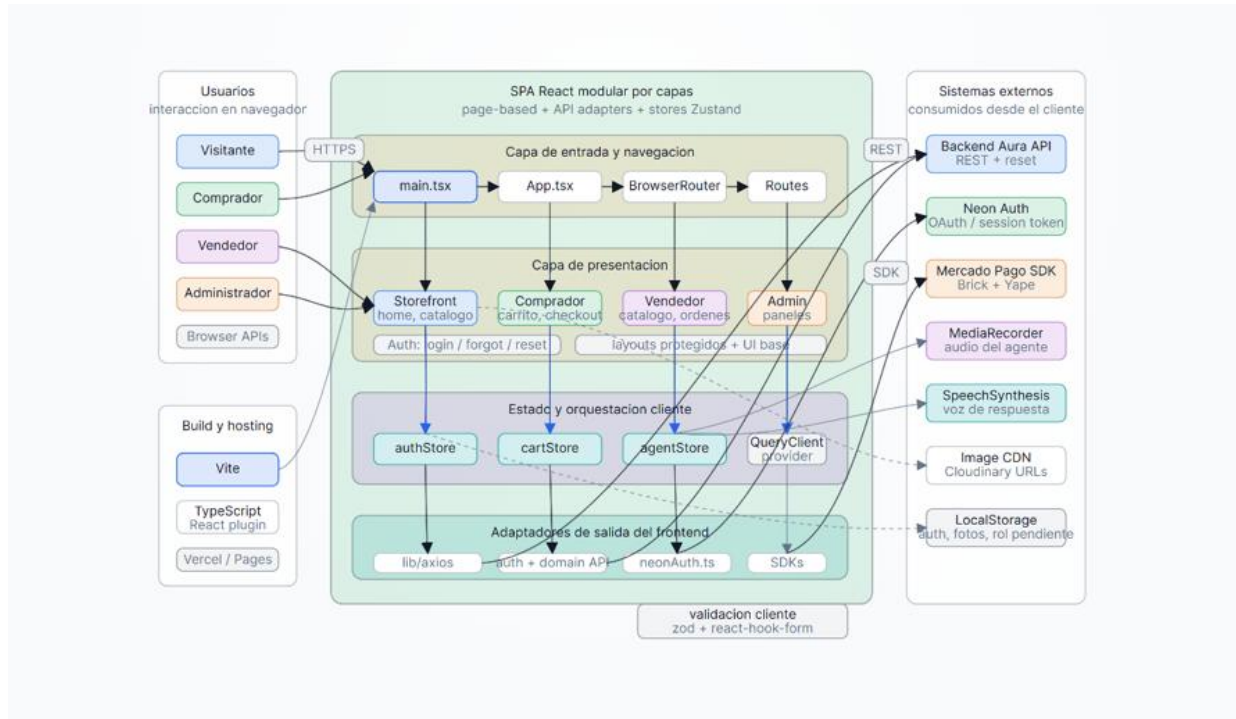


Figura A2. Arquitectura modular del frontend React. Fuente: imagen proporcionada para anexos del proyecto.

Anexo 4.3. Boundaries de backend, infraestructura y datos

La figura se incorpora como evidencia visual del diseño proporcionado. Su lectura complementa las secciones de arquitectura general, backend, frontend, persistencia e integraciones.

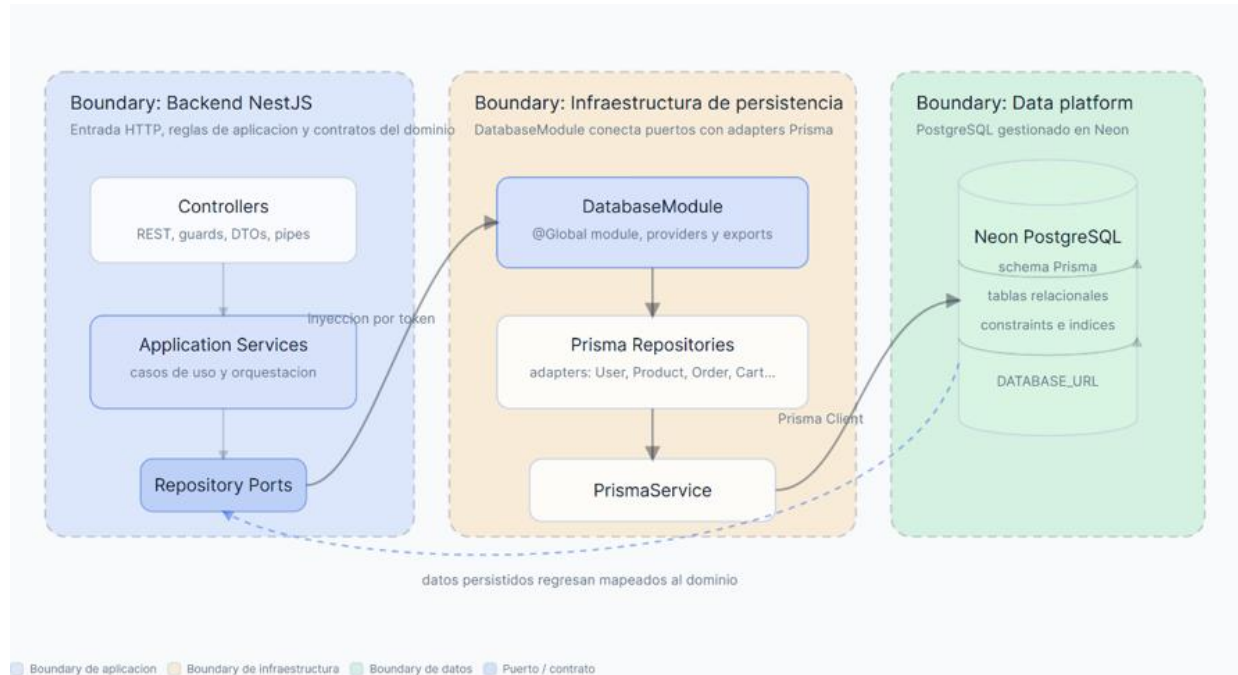


Figura A3. Boundaries de backend, infraestructura y datos. Fuente: imagen proporcionada para anexos del proyecto.

Anexo 4.4. Modelo entidad-relacion del marketplace

La figura se incorpora como evidencia visual del diseño proporcionado. Su lectura complementa las secciones de arquitectura general, backend, frontend, persistencia e integraciones.

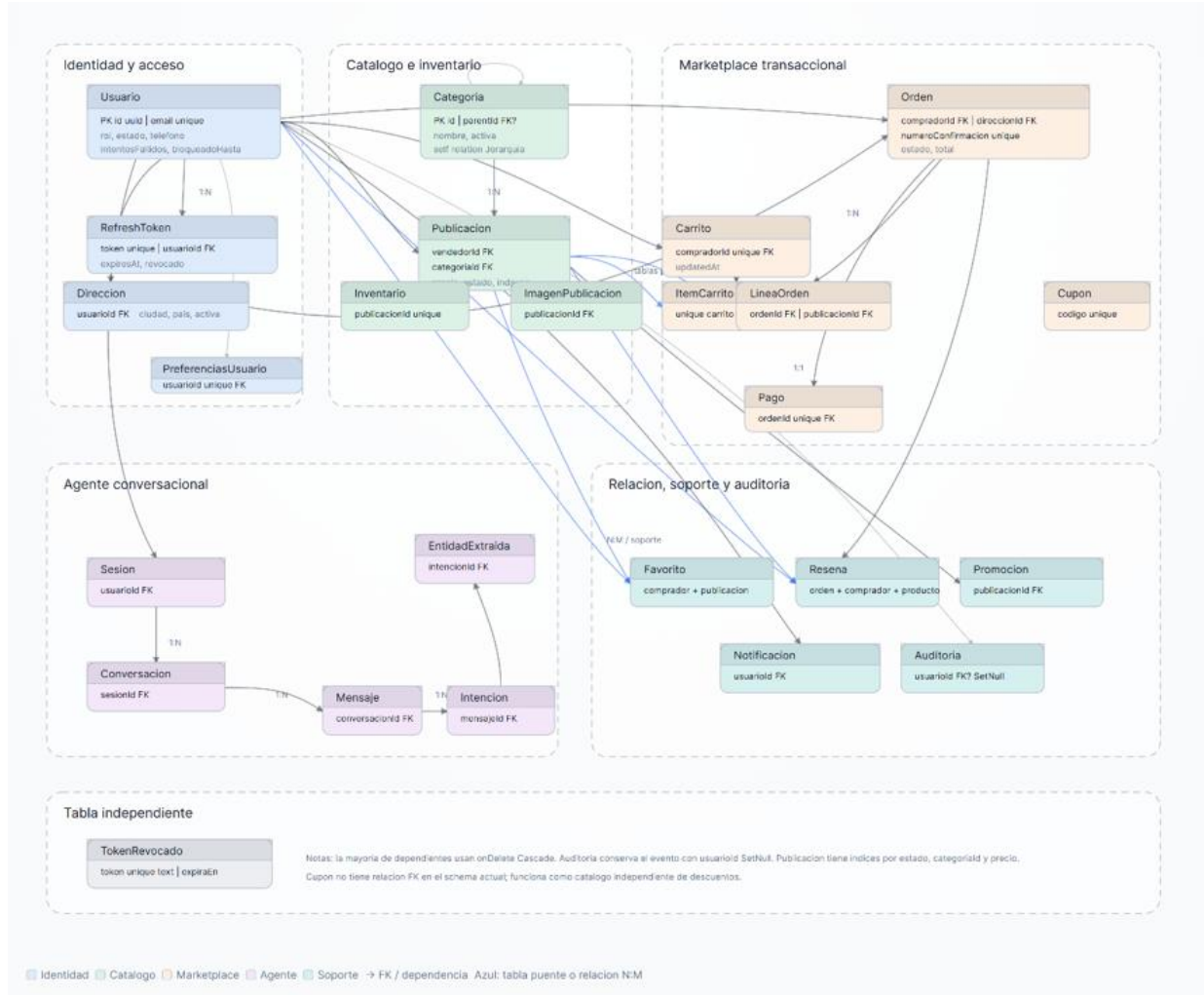


Figura A4. Modelo entidad-relacion del marketplace. Fuente: imagen proporcionada para anexos del proyecto.

Anexo 4.5. Stack tecnológico y despliegue conectado

La figura se incorpora como evidencia visual del diseño proporcionado. Su lectura complementa las secciones de arquitectura general, backend, frontend, persistencia e integraciones.

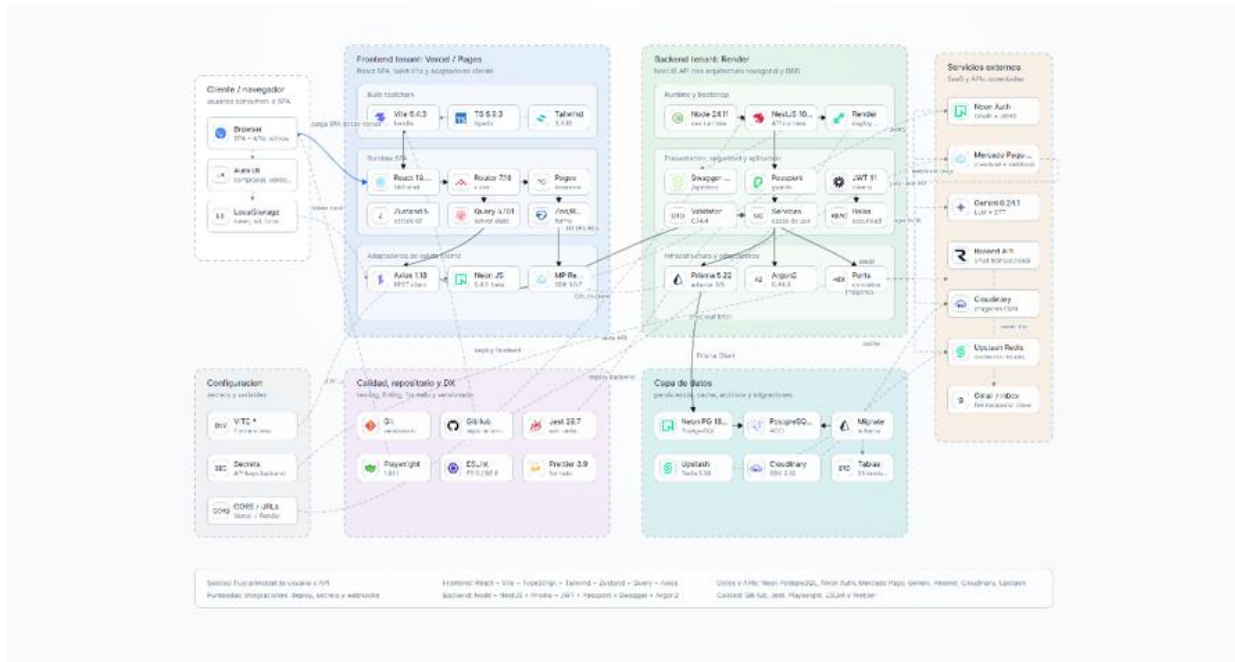


Figura A5. Stack tecnológico y despliegue conectado. Fuente: imagen proporcionada para anexos del proyecto.

Anexo 5. Contratos API principales

La tabla sintetiza la superficie REST esperada según la estructura de controladores del backend.

Modulo	Rutas representativas	Responsabilidad
Auth	/auth/register, /auth/login, /auth/refresh, /auth/verify-email	Identidad y sesiones
Products	/products, /products/:id	Catalogo y publicaciones
Cart	/cart, /cart/items	Carrito del comprador
Orders	/orders, /vendor/orders	Compra y gestión de órdenes
Payments	/payments, /payments/webhook	Pago y eventos externos
Agent	/agent/message, /agent/voice	Interacción conversacional
Admin	/admin/users, /admin/products, /admin/reports	Administración

Anexo 6. Agrupación de entidades Prisma

La agrupación evidencia que la base de datos no es una colección dispersa de tablas, sino un modelo relacional organizado por subdominios.

Subdominio	Entidades
Identidad	Usuario, RefreshToken, Direccion, PreferenciasUsuario, TokenRevocado
Catalogo	Categoria, Publicacion, Inventario, ImagenPublicacion
Marketplace	Carrito, ItemCarrito, Orden, LineaOrden, Pago, Cupon
Agente	Sesion, Conversacion, Mensaje, Intencion, EntidadExtraida
Soporte	Favorito, Resena, Promocion, Notificacion, Auditoria

Anexo 7. Estrategia de pruebas

Las pruebas se distribuyen para verificar tanto unidades aisladas como flujos completos de negocio.

Tipo de prueba	Cobertura
Unitarias backend	Servicios, controladores, DTOs, providers y utilidades
Integracion backend	Auth, RBAC, pagos, ordenes, base de datos, Redis, Cloudinary, Resend, Gemini
E2E frontend	Autenticacion, compra, vendedor, administrador y agente
Validacion manual	Diagramas, despliegue, variables y anexos

Anexo 8. Checklist de despliegue

El despliegue exige separar secretos, validar dominios, configurar CORS y comprobar endpoints de salud antes de exponer el sistema a usuarios.

Componente	Plataforma	Acciones
Backend	Render	npm ci, prisma generate, build, migrate deploy, start:prod
Frontend	Vercel / Pages	npm ci, npm run build, salida dist
Datos	Neon PostgreSQL	DATABASE_URL segura y migraciones Prisma
Imagenes	Cloudinary	Cloud name, API key, API secret y carpeta
Cache	Upstash Redis	REST URL, token y prefijo de cache

Anexo 9. Glosario tecnico

El glosario consolida terminos usados a lo largo del informe para mantener un lenguaje comun.

Termino	Definicion
Adaptador	Implementacion concreta de un puerto hacia una tecnologia externa.
Puerto	Interfaz que define una necesidad del dominio o aplicacion.
SDD	Specs Driven Development: desarrollo guiado por especificaciones versionadas y trazables.
SPA	Aplicacion web de una sola pagina que renderiza rutas en el navegador.
DTO	Objeto de transferencia de datos usado para validar entradas y salidas.
RBAC	Control de acceso basado en roles.

Anexo 10. Evidencia complementaria de diseño

En el contexto de Aura Marketplace, la evidencia complementaria numero 10 se utiliza para reforzar la relacion entre requisito, diseño, implementacion y prueba se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de anexos, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Mantiene trazabilidad con las decisiones descritas en los capitulos principales.
- Puede ampliarse con capturas, reportes de pruebas o commits especificos.
- Sirve como espacio de soporte para revisiones academicas futuras.

Anexo 11. Evidencia complementaria de diseño

En el contexto de Aura Marketplace, la evidencia complementaria numero 11 se utiliza para reforzar la relacion entre requisito, diseño, implementacion y prueba se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de anexos, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Mantiene trazabilidad con las decisiones descritas en los capitulos principales.
- Puede ampliarse con capturas, reportes de pruebas o commits especificos.
- Sirve como espacio de soporte para revisiones academicas futuras.

Anexo 12. Evidencia complementaria de diseño

En el contexto de Aura Marketplace, la evidencia complementaria numero 12 se utiliza para reforzar la relacion entre requisito, diseño, implementacion y prueba se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de anexos, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Mantiene trazabilidad con las decisiones descritas en los capitulos principales.
- Puede ampliarse con capturas, reportes de pruebas o commits especificos.
- Sirve como espacio de soporte para revisiones academicas futuras.

Anexo 13. Evidencia complementaria de diseño

En el contexto de Aura Marketplace, la evidencia complementaria numero 13 se utiliza para reforzar la relacion entre requisito, diseño, implementacion y prueba se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de anexos, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Mantiene trazabilidad con las decisiones descritas en los capitulos principales.
- Puede ampliarse con capturas, reportes de pruebas o commits especificos.
- Sirve como espacio de soporte para revisiones academicas futuras.

Anexo 14. Evidencia complementaria de diseño

En el contexto de Aura Marketplace, la evidencia complementaria numero 14 se utiliza para reforzar la relacion entre requisito, diseño, implementacion y prueba se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de anexos, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Mantiene trazabilidad con las decisiones descritas en los capitulos principales.
- Puede ampliarse con capturas, reportes de pruebas o commits especificos.
- Sirve como espacio de soporte para revisiones academicas futuras.

Anexo 15. Evidencia complementaria de diseño

En el contexto de Aura Marketplace, la evidencia complementaria numero 15 se utiliza para reforzar la relacion entre requisito, diseño, implementacion y prueba se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de anexos, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Mantiene trazabilidad con las decisiones descritas en los capitulos principales.
- Puede ampliarse con capturas, reportes de pruebas o commits especificos.
- Sirve como espacio de soporte para revisiones academicas futuras.

Anexo 16. Evidencia complementaria de diseño

En el contexto de Aura Marketplace, la evidencia complementaria numero 16 se utiliza para reforzar la relacion entre requisito, diseño, implementacion y prueba se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de anexos, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Mantiene trazabilidad con las decisiones descritas en los capitulos principales.
- Puede ampliarse con capturas, reportes de pruebas o commits especificos.
- Sirve como espacio de soporte para revisiones academicas futuras.

Anexo 17. Evidencia complementaria de diseño

En el contexto de Aura Marketplace, la evidencia complementaria numero 17 se utiliza para reforzar la relacion entre requisito, diseño, implementacion y prueba se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de anexos, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Mantiene trazabilidad con las decisiones descritas en los capitulos principales.
- Puede ampliarse con capturas, reportes de pruebas o commits especificos.
- Sirve como espacio de soporte para revisiones academicas futuras.

Anexo 18. Evidencia complementaria de diseño

En el contexto de Aura Marketplace, la evidencia complementaria numero 18 se utiliza para reforzar la relacion entre requisito, diseño, implementacion y prueba se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de anexos, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Mantiene trazabilidad con las decisiones descritas en los capitulos principales.
- Puede ampliarse con capturas, reportes de pruebas o commits especificos.
- Sirve como espacio de soporte para revisiones academicas futuras.

Anexo 19. Evidencia complementaria de diseño

En el contexto de Aura Marketplace, la evidencia complementaria numero 19 se utiliza para reforzar la relacion entre requisito, diseño, implementacion y prueba se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de anexos, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Mantiene trazabilidad con las decisiones descritas en los capitulos principales.
- Puede ampliarse con capturas, reportes de pruebas o commits especificos.
- Sirve como espacio de soporte para revisiones academicas futuras.

Anexo 20. Evidencia complementaria de diseño

En el contexto de Aura Marketplace, la evidencia complementaria numero 20 se utiliza para reforzar la relacion entre requisito, diseño, implementacion y prueba se entiende como un componente critico del ciclo de vida del sistema, porque permite conectar las decisiones tecnicas con los objetivos del negocio, la experiencia del usuario y la sostenibilidad del producto. Su estudio no se limita a describir una tecnologia; tambien explica por que esa tecnologia se adopta y que consecuencias tiene sobre la calidad final.

Desde la perspectiva de anexos, el proyecto requiere separar los conceptos teoricos de las decisiones de implementacion. Esta separacion evita que el documento sea una simple descripcion de pantallas o archivos y permite demostrar que cada modulo responde a principios de ingenieria: bajo acoplamiento, alta cohesion, trazabilidad, seguridad por diseño y validacion continua.

La aplicacion concreta se observa en la integracion entre una SPA React, un backend NestJS organizado por capas L01 a L05, una base de datos PostgreSQL administrada mediante Prisma y servicios externos como Neon Auth, Mercado Pago, Resend, Cloudinary, Upstash Redis y Gemini AI. La arquitectura se diseña para que esas dependencias sean reemplazables y no contaminen el nucleo del dominio.

- Mantiene trazabilidad con las decisiones descritas en los capitulos principales.
- Puede ampliarse con capturas, reportes de pruebas o commits especificos.
- Sirve como espacio de soporte para revisiones academicas futuras.